

GDA API Gateway Plan

Date: April 27, 2026

Short answer

Yes: GDA should keep React as the main UI and n8n as the automation engine, but React should not talk directly to dozens of n8n webhook details forever. The stable architecture is:

```
React UI → GDA API Gateway → n8n workflows → Postgres / external services
```

Today the nginx /api/... proxy is the emergency bridge. It works and it solved the key-in-browser problem. The next mature step is a small GDA API Gateway that owns auth, response normalization, dry-run enforcement, audit logging, and QA Center endpoints.

Why this helps

Current pain	Gateway fix
React has to know many n8n paths	React calls stable business endpoints.
n8n response shapes vary	Gateway normalizes responses into one envelope.
Auth keys drift between React, nginx, n8n, Retool	Gateway owns backend secrets; React gets none.
Hard to test what is safe	Gateway enforces read-only, dry-run, or approval rules.
Hard for Shawn to read failures	Gateway converts workflow failures into plain-English status.
Retool duplicates React	Retool can be retired; React becomes the only operator UI.

Keep vs replace

Layer	Keep?	Role
React	Yes	Main GDA Command UI and future QA Center.
n8n	Yes	Automation/workflow engine, scheduled jobs, integrations.
Postgres	Yes	System of record.
Retool	Retire/archive	Temporary replacement experiment; not needed long-term.
nginx /api proxy	Keep short-term	Secure bridge while gateway is built.
New API Gateway	Add next	Stable backend contract between React and n8n.

Recommended gateway endpoints

Endpoint	Purpose	Safety
GET /api/qa/health	Run QA Center read-only checks and return summary.	Read-only
POST /api/qa/dry-run	Run approved dry-run tests only.	Dry-run only
GET /api/workflows/registry	Return workflow registry baseline.	Read-only
GET /api/failures/latest	Return latest n8n failures with names and reasons.	Read-only
POST /api/opportunities/save	Save opp through n8n, with dry-run support.	Dry-run/test-row/approved
POST /api/risks	Risk create/update/list through n8n.	Dry-run/test-row/approved
POST /api/capture-plan/preview	Preview capture plan prompt/inputs without save.	Dry-run/preview
POST /api/deep-research/preview	Validate request and estimate workflow before paid call.	Preview/approval
POST /api/competitors/crawl-preview	Show what would be crawled before paid/external call.	Preview/approval

Implementation options

Option A: Small Node/Express gateway beside React

Best fit if the frontend container can run Node. Pros: easy JavaScript, easy JSON normalization, easy QA Center routes.

Option B: FastAPI gateway

Best fit if you want stronger typing and Python utilities. Pros: clean validation with Pydantic, straightforward scheduled/reporting scripts.

Option C: Continue nginx-only proxy

Good emergency bridge, not enough long-term. nginx can inject keys and route paths, but it cannot easily normalize workflow responses, enforce dry-run policy, or explain failures.

Recommended path

Use Option A: a small Node/Express GDA API Gateway.

Phase 1: Add gateway read-only endpoints only.

- /api/qa/health
- /api/qa/dry-run

- `/api/workflows/registry`
- `/api/failures/latest`

Phase 2: Move high-value business routes behind gateway.

- Dashboard
- Opportunity tracker
- Semantic search
- Risk register
- Capture plan

Phase 3: Add human approval for dangerous actions.

- Deep research paid calls
- Competitor crawler external calls
- Telegram/email sends
- Deploy/fix workflows
- Bulk ingest

Security rules

- No n8n API key in React.
- No webhook key in React.
- No QA/fix agent key in React.
- All secrets live server-side only.
- Gateway logs every write/dry-run/approval action.
- Gateway rejects write-capable requests unless one is true:
 - `dryRun: true`
 - clearly marked test row with cleanup rule
 - human approval token

What this means practically

Do not rebuild everything from scratch. The sound rescue path is:

1. Keep React.
2. Retire Retool.
3. Keep n8n.
4. Keep the current nginx proxy as the bridge.
5. Add QA Center now.
6. Add a small API Gateway next.
7. Move one workflow family at a time behind the gateway.

8. Only then do deeper UX/business feature work.

Immediate next build

After the QA Center source is deployed, build the gateway v1 with two routes:

```
GET /api/qa/health  
POST /api/qa/dry-run
```

Those routes should call the same checks we already validated and return one plain-English summary for the dashboard.